THE UNIVERSITY OF DA NANG

UNIVERSITY OF SCIENCE AND TECHNOLOGY

FACULTY OF ADVANCED SCIENCE AND TECHNOLOGY

COURSE PROJECTS

COURSE'S TITLE: ENGINEERING COMPUTATION

PROJECT'S TITLE:

FACE DETECTION USING MATLAB

Supervisor: **Dr. Doan Quoc Khoa**

Students: **Nguyễn Tiến Hưng**

Dương Vân Trường

Nguyễn Minh Quang

Trần Đình Toàn Thắng

Nguyễn Lê Hoài Nam

Nguyễn Hữu Trung

Nguyễn Trịnh Hoàng Quân

Lê Việt Nhật

Class: **24ECE**

Da Nang, 4/2026

TABLE OF CONTENTS

Introduction

Chapter 1. Theoretical Background……1

1.1. Basic theory ….....….....….....1

1.1.1. *Image Representation*

1.1.2. *Grayscale Conversion*

1.2. Face Detection Algorithm

1.2.1. *Haar Cascade Classifier*

1.2.2. *Detection Parameters*

1.2.3. Bounding Box Representation

Chapter 2. Program Implementation

2.1. System Structure

2.2. Program Workflow

2.3 Function Description

2.3.1 *Function: load_image*

2.3.2 *Function: validate_image*

2.3.3 *Function: resize_image*

2.3.4 *Function: convert_gray*

2.3.5 *Function: create_detector*

2.3.6 *Function: detect_faces*

2.3.7 *Function: count_faces*

2.3.8 Function: draw_faces

2.3.9 Function: display_result

2.4 Results

2.5 Advantages-Disadvantages and Future Development

2.5.1 *Advantages*

2.5.2 *Disadvantages*

2.5.3 *Future Development*

2.6 Conclusion

INTRODUCTION

I. Introduction:

- Face Detection is one of the most fundamental and crucial tasks in the field of Computer Vision. The goal of this challenge is to find the positions and sizes of human faces in a digital image, regardless of conflicting elements like illumination, camera angles, or the number of people.
- This initiative concentrates on developing a system for detecting and counting faces solely utilizing the MATLAB programming language. Rather than strictly relying on preset image processing tools, the project delves into the core principles by custom-building a function to convert color images to grayscale (an algorithm that extracts the **R, G, B** color channels using the ratio **0.299R + 0.587G + 0.114B**). The core of the system is the Viola-Jones object recognition method **[1]**, executed using the `vision.CascadeObjectDetector` tool. The system is designed to be processed and tested on real-world group photos with varying crowd densities (such as sports events or faculty/school commemorative photos), which requires the ability to fine-tune high-sensitivity parameters to ensure no subjects are missed.

II. Project Objectives:

- The project is organized to reach the following primary goals:

- **Building a complete Image Processing Pipeline**: Implement an automated workflow from image preprocessing (reading the image, converting to grayscale using the custom `convert_gray.m` function), object detection (`detect_faces.m`), to post-processing and visual display (display_result.m).
- **Model Hyperparameter Tuning**: Empirically investigate and fine-tune the structural parameters of the `vision.CascadeObjectDetector` classifier, specifically including the *MergeThreshold*, *ScaleFactor*, and *MinSize*. The goal is to establish an optimal set of weights that enhances the algorithm's sensitivity when processing image data with dense crowd concentrations and small facial pixel sizes.
- **Spatial Quantification and Statistics**: Develop an algorithm to map and count the number of bounding boxes obtained from the prediction model, providing an accurate quantitative database for statistical applications.
- **Empirical Evaluation**: Test the robustness of the algorithm on highly specific real-world image datasets (sports events, outdoor group photos) with complex background noise and varying image acquisition conditions.

III. Potential Applications:

- The results of this study provide an algorithmic foundation and technical solutions for specialized practical applications:

- **Crowd Analysis & Monitoring Systems**: The algorithm can be integrated into closed-circuit television (CCTV) networks for density estimation and autonomous

quantification of individuals within a given space (e.g., lecture halls, stadiums), facilitating crowd flow management.

- **Biometric Pre-processing Module**: In Face Recognition systems, the face detection module acts as an essential spatial filter. It helps localize and extract the Region of Interest (ROI), eliminating surrounding background noise before feeding the data into Artificial Neural Networks (e.g., CNNs) for classification.
- **Adaptive Optics Control**: Applied in the firmware of smart imaging devices to automatically direct the auto-focus system and perform local auto-exposure calibration based on the centroid coordinates of faces interpolated from the image space.

Chapter 1: Theoretical Background

1.1. Basic Theory

1.1.1. Image Representation

1. Digital Image

- **Definition:** A digital image is a numerical representation of a real-world visual, divided into a grid of tiny elements called pixels (picture elements).
- **Nature of the data:** Each pixel stores information about the light intensity or color at a specific location. The more pixels there are, the higher the image resolution and the sharper the details.
- **In image processing:** Instead of seeing with the human eye, a computer "sees" an image as a set of discrete numbers. Image processing is essentially performing mathematical operations on these numbers.

2. Image Matrix

- Since MATLAB is fundamentally designed to work with matrices (Matrix Laboratory - a core concept from Chapter 2), every image loaded into the software is converted into mathematical matrices. The structure of the matrix depends on the image type:

● Grayscale Image: Stored as a 2D matrix of size M x N. Each element I(x,y) holds an integer value ranging from 0 (absolute black) to 255 (absolute white), representing the brightness level of the pixel at those specific coordinates.

This numerical range from 0 to 255 is officially known as **Pixel Intensity** or **Grayscale value**.

- **Hardware Nature (8-bit):** Each pixel in a standard grayscale image is allocated a memory capacity of 8 bits (which equals 1 byte) by the computer.
- **Mathematics:** According to the binary system, 8 bits can generate exactly 2^8 = 256 different states or values.
- **Value Range:** Since computers always start counting from 0, this set of 256 values ranges from 0 to 255.

Grayscale Image (2D Matrix: M x N)

- Unlike color images, grayscale images have their color vectors completely removed, retaining only a single piece of information at each coordinate: the Pixel Intensity.
- Since there is only one intensity value per pixel, the data structure of a grayscale image is reduced to a flat 2-dimensional matrix (2D Matrix).
- The dimensions of this matrix are simply M x N. Each element I(x,y) in the matrix contains a discrete integer (ranging from 0 to 255) that reflects the brightness level of the pixel at that specific location.

RGB Color Image (3D Matrix: M x N x 3)

Because color images contain complex information, they are defined as a 3-dimensional matrix space (3D Matrix) **[4]**. The notation M x N x 3 represents:

- M (Rows): The number of rows in the matrix, corresponding to the height of the image (measured in pixels).
- N (Columns): The number of columns in the matrix, corresponding to the width of the image. This M x N grid defines the resolution of the image space.
- 3 Color Channels: The third dimension (the depth of the matrix) represents the 3 independent color channels that construct an RGB image. It consists of three 2D matrix layers stacked on top of each other: a layer representing the intensity of Red, a layer for Green, and a layer for Blue.
- RGB Image (Color Image): More complex, it is represented by a 3D matrix of size M x N x 3. Essentially, these are three 2D matrices stacked together, representing the three primary color channels:
    + Channel 1: Red
    + Channel 2: Green
    + Channel 3: Blue

1.1.2. Grayscale Conversion

- To simplify processing, RGB images are converted into grayscale using:

$$\textbf{Gray} = 0.299R + 0.587G + 0.144B \ [2]$$

This formula originates from extensive scientific experiments conducted on a large population. Scientists performed empirical measurements on thousands of human eyes to determine which light colors we are most sensitive to. As a result, they discovered the 'golden ratio' of the human retina:

+ The human eye is most sensitive to Green light (accounting for approximately 58.7%).
+ The second highest sensitivity is to Red light (accounting for about 29.9%).
+ The eye is least sensitive to Blue light (accounting for only 11.4%).

1.2. Face Detection Algorithm

1.2.1. Haar Cascade Classifier

- The program uses MATLAB's built-in function: **vision.CascadeObjectDetector**

→ This method is based on trained classifiers that detect facial features such as eyes, nose, and mouth.

- The Haar Cascade Classifier is an object detection algorithm widely used in face detection that works by scanning an image and identifying patterns that look like a human face. **[1]**
- Its operation can be understood in **4 main steps**:

a) Haar-like Features (Detecting Patterns)

- The algorithm uses simple features called **Haar-like features**.
- These features compare the brightness between different regions of an image:
- The eye region is usually darker than the cheeks
- The nose bridge is usually brighter than the sides

Instead of understanding the image like a human, the algorithm only checks **light and dark patterns**.

b) Integral Image (Speed Optimization)

- To make the process very fast, the algorithm converts the image into an **integral image** and the purpose are:
- Quickly calculate pixel sums in any rectangular area
- Reduce computation time significantly

→ This allows the detector to scan the image in real-time.

c) AdaBoost (Selecting Important Features)

- There are thousands of possible features, but not all are useful.
- AdaBoost is used to:
- Select the most important features
- Combine weak classifiers into a strong classifier

d) Cascade Structure (Multi-stage Filtering)

The classifier is organized as a **cascade of stages**.

- Stage 1: Quickly removes regions that are clearly not faces
- Stage 2: Checks more carefully
- Stage 3: Even stricter checking

→ Only regions that pass all stages are considered faces.

1.2.2. Detection Parameters

1. MergeThreshold

- **Definition:** MergeThreshold is a parameter in the MATLAB function vision.CascadeObjectDetector that controls how many detection results must overlap before they are considered a valid face.
- **Working Principle:** During the detection process, the algorithm scans the image at multiple positions and scales. This often produces multiple overlapping bounding boxes for the same face. MergeThreshold determines how many of these overlapping detections are required to confirm a face.

- A **higher value** means stricter detection (fewer faces detected)
- A **lower value** means more sensitive detection (more faces detected)

Effect on performance:

Value Range	Effect
2 – 4	High sensitivity, detects many faces, but increases false positives
5 – 6	Balanced detection and accuracy
7 – 10	Strict detection, fewer false positives, but may miss faces

Value used in this project: MergeThreshold = 5;

Conclusion: The value of 5 is selected as a compromise between sensitivity and accuracy. It is lower than the default value (7), which improves the ability to detect faces. It still maintains enough strictness to reduce false detections. Compared to lower values (e.g., 3 or 4), it produces more stable and reliable results.

➢ The chosen value ensures that the system detects most faces while keeping the number of false positives at an acceptable level.

2. ScaleFactor

- **Definition:** ScaleFactor controls how the detection window is scaled when scanning the image for faces.
- **Working Principle:** The face detector searches for faces of different sizes by resizing the detection window. ScaleFactor determines the step size between these scales.

- **Smaller values** → finer scanning → more accurate detection
- **Larger values** → faster processing → less accurate

Effect on performance:

Value	Effect
1.05	Very high accuracy, slower processing
1.1	Balanced
1.2	Faster but may miss faces

Value used in this project: ScaleFactor = 1.1;

Conclusion:

The value of 1.1 is selected as a balance between detection accuracy and processing speed:

- Provides sufficient scanning detail to detect faces of different sizes
- Faster than smaller values such as 1.05
- Reduces computational cost while maintaining acceptable accuracy

Compared to 1.05:

- Slightly less accurate in detecting very small or rotated faces
- Faster and more efficient.
- ➢ The selected value ensures stable and efficient performance, making it suitable for real-time or standard applications where both speed and accuracy are important.

3. MinSize

- **Definition:** MinSize defines the minimum size of a face that can be detected.
- **Working principle:** Faces smaller than this size will be ignored during detection.

This helps:

- Reduce noise
- Avoid detecting very small non-face objects

Effect on performance:

Value	Effect
Small (20×20)	Detects small faces but may increase noise
Large (50×50)	Ignores small faces but more stable

Value used in this project: MinSize = [20 20];

Conclusion: A small value is chosen to allow detection of smaller faces while maintaining acceptable accuracy.

4. Effect of Parameter Adjustment

To evaluate the performance of the face detection system, experiments were conducted using different parameter settings.

Case 1: Default Parameters

- MergeThreshold = 7
- ScaleFactor = 1.05
- MinSize = [50 50]

Result:

- ➢ Only clearly visible frontal faces were detected
- ➢ Some faces were missed
- ➢ Detection was stable but less sensitive

Case 2: Adjusted Parameters (Proposed Method)

- MergeThreshold = 5
- ScaleFactor = 1.1
- MinSize = [20 20]

Result:

- ➢ More faces were successfully detected
- ➢ Smaller faces could be identified
- ➢ Slight increase in false detections

Comparison Summary:

Criteria	Default Parameters	Adjusted Parameters
Detection rate	Medium	High
Ability to detect small faces	Low	High
False positives	Low	Medium
Processing speed	Fast	Slightly slower

Conclusion: Reducing the MergeThreshold and MinSize improves detection sensitivity, allowing the system to detect more faces, especially smaller or less clear ones. However, this also increases the possibility of false detections.

Table 1.2 Technical features of the face detection system

No.	Parameter	Value	Description
1	Detection Method	Haar Cascade	Algorithm used for face detection
2	MATLAB Function	vision.CascadeObject Detector	Built-in detector function
3	MergeThreshold	5	Controls detection sensitivity
4	ScaleFactor	1.1	Controls scanning scale

			step
5	Min Face Size	20 × 20 pixels	Minimum detectable face
6	Input Image Type	RGB Image	Image format requirement
7	Output	Bounding Box	Face location in image
8	Programming Language	MATLAB	Implementation platform

1.2.3. Bounding Box Representation

1. Concept:

- In Pattern Recognition and Computer Vision, a Bounding Box is a geometric representation used to define the spatial location and boundary of a target object (in this case, a human face) within a 2D image plane.

2. Data Structure and Coordinate System

- Within the MATLAB computing environment, the output of the object detection algorithm (via the `vision.CascadeObjectDetector` object) is a Bounding Box matrix **[3]**. Each detected object is specified by a mathematical vector consisting of four spatial values: ***[x, y, w, h]***

+ ***x, y***: The Cartesian coordinates of the pixel located at the top-left corner of the bounding box. The origin **(0,0)**, according to digital image conventions, is calculated from the top-left corner of the entire image.
+ ***w* (Width)**: The horizontal dimension of the bounding box, representing the number of pixels extending along the x-axis.
+ ***h* (Height)**: The vertical dimension of the bounding box, representing the number of pixels extending along the y-axis.

3. Application in the Processing Pipeline

- When the Viola-Jones algorithm scans the image and identifies a region containing Haar-like features that surpass the classification threshold of the Cascade Classifier, it automatically calculates and returns a matrix containing these ***[x, y, w, h]*** vectors.
- This coordinate data is a mandatory foundation for subsequent post-processing steps, specifically:

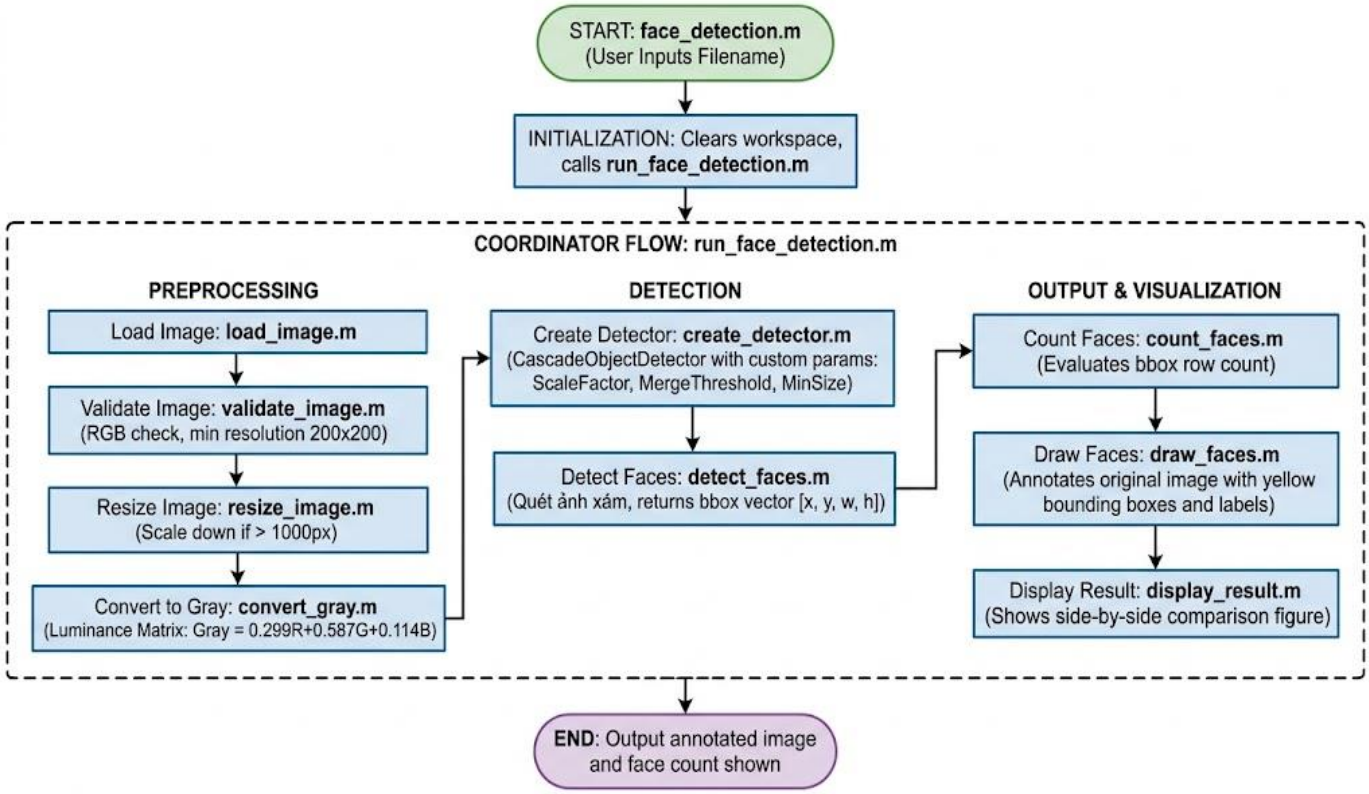
+ Visually locating and marking the facial region on the original image using graphical functions (e.g., insertShape).
+ Extracting the Region of Interest (ROI) for further analytical algorithms without the need to reprocess the entire original image, thereby optimizing computational workload.

Chương 2: PROGRAM IMPLEMENTATION

2.1. System Structure

- The system operates as a sequential pipeline managed by run_face_detection.m:
  + **Initialization:** face_detection.m prompts the user for the image filename and triggers the main workflow.
  + **Preprocessing:** The image is loaded and validated (load_image.m, validate_image.m), resized to optimize performance (resize_image.m), and converted to grayscale (convert_gray.m).
  + **Detection:** A customized Viola-Jones cascade detector (create_detector.m) scans the grayscale image to extract the bounding box coordinates of faces (detect_faces.m).
  + **Output:** The system counts the detected faces (count_faces.m), overlays yellow bounding boxes onto the original image (draw_faces.m), and displays the final visual result (display_result.m).

2.2. Program Workflow



- **Main Function Analysis (run_face_detection.m)**
  + The run_face_detection function acts as the central coordinator of the system. Instead of performing calculations directly, it sequentially executes specialized modules to form a complete pipeline:
  + **Preparation:** Calls load_image and validate_image to import and verify the image.
  + **Preprocessing:** Uses resize_image and convert_gray to optimize the image for the algorithm.
  + **Detection:** Initializes the Haar cascade (create_detector) and extracts bounding box coordinates (detect_faces).
  + **Visualization:** Forwards data to count_faces, draw_faces, and display_result to output the final annotated image.

2.3 Function Description

2.3.1 Function: load_image

- [filename, pathname] = uigetfile({'*.jpg;*.png;*.jpeg'}, 'Select an image');

Triggers a System Call to open the native OS file browser. The cell array acts as a System-level Filter, preventing the user from selecting incompatible file types (like PDFs) and mitigating errors proactively.

- if isequal(filename, 0)

When the user clicks "Cancel", filename is assigned a numeric double value of 0 (not an empty string). Using isequal safely compares both value and data type, avoiding a Type Mismatch error. Returning an empty array ([]) acts as a flag to safely halt the main pipeline, preventing Null Pointer Exceptions downstream.
- `fullPath = fullfile(pathname, filename);`

Automatically detects the host OS (Windows vs. UNIX-based) and inserts the correct directory separator (\ or /), ensuring cross-platform compatibility.
– `img = imread(fullPath);`
Reads the byte stream from the hard drive, decodes the compression (e.g., JPEG's DCT), and allocates RAM to construct a 3D matrix (H x W x C) of uint8 values. This transforms the file into a mathematical tensor ready for computation.

2.3.2 Function: validate_Image

- **Purpose**: a **Precondition Checker**. Its job is to evaluate whether the image matrix meets the mathematical and physical formats required for the Cascade algorithm to function stably, preventing runtime crashes or "Garbage In, Garbage Out" scenarios.
- `if size(img, 3) ~= 3`

Data Structure Basis: A standard color image is loaded as a 3D tensor with dimensions H x W x C. The 3 represents the RGB channels
Risk: The subsequent `convert_gray` function has a hardcoded linear formula for exactly 3 matrices. If the user inputs a grayscale image (1 channel) or a transparent PNG (4 channels), the program will throw an "Index out of bounds" error. The `warning` serves as a soft alert..
- `if h < 200 || w < 200`

Low Spatial Frequency Consequences: If the overall image is under 200x200 pixels, a face in a crowd might only occupy 10 x 10 pixels. At this low sampling rate, vital high-frequency features (like the contrast between dark eye sockets and a bright nose bridge) are lost to pixelation/blur. The classifier will fail to find these patterns, resulting in False Negatives.

2.3.3 Function: resize_image

- **Purpose**: `resize_image` function acts as a **pre-processing filter** to standardize all images to a "golden size" before feeding them into the detector. If the image is too small, faces become blurry and lose defining features. If the input is an ultra-high-resolution image (e.g., a 4K image), the volume of convolutional calculations will explode, causing the program to run extremely slowly.
- `max_size = 1000;`

Defines the maximum allowable length for the image's longest edge. Value 1000 to keep it large to retain small detail of faces and prevent CPU/RAM overload.
-`[h, w, ~] = size(img);`

Extracts the image's height (h) and width (w), scaling only affects 2D spatial coordinates

- `scale = min(max_size / max(h, w), 1);`

`+max(h, w)`

Find the longest side of the image.

`+ max_size / max(h, w)`

Formula to calculate the scale to scale down the image to exactly 1000 pixels.

`+min(.....,1);`

Function strictly limits the scaling factor to a maximum of 1.0. This guarantees the algorithm **only downscales** oversized images and never upscales smaller ones

`+ img_out = imresize(img, scale);`

After calculating the optimal scale factor, the imresize function formally changes the pixel count of the image matrix.

2.3.4 Function: convert_gray

- **Purpose**: The primary task of this function is to convert a color image (RGB) into a grayscale image using a weighted sum method.
- The input image is first converted to double precision:

  img = double(img); (this step ensures accurate mathematical computation when combining color channels)

- A color image is stored as a **3D matrix (M × N × 3)**.
- In this matrix:

  M, N: Represent the height and width of the image.

  3 channels: Correspond to Red (R), Green (G), and Blue (B).

  The function separates the three color channels:

  `img(:,:,1)` → Red channel

  `img(:,:,2)` → Green channel

  `img(:,:,3)` → Blue channel

- The grayscale image is computed using the standard luminance formula:

  Gray = 0.299*R + 0.587*G + 0.114*B;

- These weights reflect human visual sensitivity:
  - Green contributes the most.

    - Red contributes moderately.
    - Blue contributes the least.
- The result is then converted back to **uint8 format**:

  gray = uint8(gray); (ensures the output image has valid pixel values in the range **0–255**)

2.3.5 Function: create_detector

- **Purpose:** The primary task of this function is to initialize a face detector used to identify human faces in an image.
- **The detector** is created using:

  detector = vision.CascadeObjectDetector();

  - This is a built-in MATLAB object that uses the Haar Cascade algorithm
  - It is pre-trained to detect human faces
- After initialization, the detector parameters are adjusted to improve performance:

  1. MergeThreshold: detector.MergeThreshold = 5;

  - Controls the sensitivity of merging detected regions
  - Lower values → more detections (may include false positives)
  - Higher values → fewer false detections but may miss some faces

  2. ScaleFactor: detector.ScaleFactor = 1.1;

- Determines how much the image is scaled during scanning
- Values closer to 1 → more precise detection but slower processing
- 1.1 provides a good balance between accuracy and speed

  3. MinSize: detector.MinSize = [20 20];

- Specifies the minimum size of detectable faces
- Helps eliminate small noise regions that are not faces

2.3.6 Function: detect_faces

- **Purpose:** The primary task of this function is to initialize a face detection model used to identify human faces in an image.

The detector is created using MATLAB's built-in object:

function bbox = detect_faces(detector, img)

bbox = step(detector, img);

- This object uses a pre-trained Haar Cascade classifier to detect faces.
- The detector scans the image at multiple scales and positions to locate regions that resemble facial features.
- The output of this function is a detector object that will be used in later steps.

2.3.7 Function: count_faces

- **Purpose**: The primary task of this function is to convert geometric data (the coordinates of the bounding boxes surrounding the faces)
- bbox stands for **Bounding Box**.
- This spatial data is stored in MATLAB as a matrix with dimensions N x 4. In this matrix:
+ **N (Rows):** Corresponds to the number of detected faces.
+ **4 (Columns):** Each face is defined by 4 geometric parameters in the order [x, y, width, height].
- `size(A, dim)` function is used to get the length of matrix `A` along dimension `dim`.
+ `dim = 1`: Returns the number of **rows**.
+ `dim = 2`: Returns the number of **columns**.
- Each row in the `bbox` matrix represents the coordinates of exactly **one** face. Determining the total number of faces simply requires counting the rows of this matrix.

2.3.8 Function: draw_faces

- **Purpose**: The primary task of this function is to visualize detected faces by drawing rectangles around them.

The function takes:

- This data is stored in MATLAB as a matrix with dimensions **N × 4**.

In this matrix:

+ **N (Rows)**: Number of detected faces
+ **4 (Columns)**: Each face is defined by `[x, y, width, height]`
- The function first draws rectangles around detected faces:

```
img_out = insertShape(img, 'Rectangle', bbox, ...
'Color', 'yellow', 'LineWidth', 3);
```

`'Rectangle'`: Specifies the shape to draw

`bbox`: Position and size of each face

`'Color', 'yellow'`: Sets the rectangle color

`'LineWidth', 3`: Controls the thickness of the box

Next, the function checks whether any faces are detected:

if ~isempty(bbox)

`isempty(bbox)` returns true

if no faces are found ~ means NOT

→ ensures labels are added only when faces exist

- If faces are detected, labels are added at the top-left corner of each bounding box:

```
img_out = insertText(img_out, bbox(:,1:2), ...
'Face Detected', ...
'BoxColor', 'yellow', ...
'TextColor', 'black');
```

`bbox(:,1:2)` extracts the **(x, y)** coordinates of each face

`'Face Detected'`: Text label displayed

`'BoxColor'`: Background color of the text

`'TextColor'`: Color of the text

2.3.9 Function: display_result

- **Purpose**: The primary task of this function is to display the original image and the processed image (with detected faces), as well as to show the number of detected faces.
- The function creates a new figure window:

  figure (This opens a new graphical window in MATLAB)

- Display the original image on the left:

```
subplot(1,2,1)

imshow(original_img)

title('Original Image')
```

- **`subplot(1,2,1)`:** Divides the window into 1 row and 2 columns → selects the first position.
- **`imshow`:** Displays the image.
- **`title`:** Adds a title to the image.

- Display the processed image on the right:

```
subplot(1,2,2)

imshow(processed_img)

title(['Detected Regions: ' num2str(num_faces)])
```

  - `subplot(1,2,2)`: Selects the second position.
  - Displays the image with detected faces.
  - `num2str(num_faces)`: Converts the number into a string for display.
  -

- Print the number of detected faces in the Command Window:

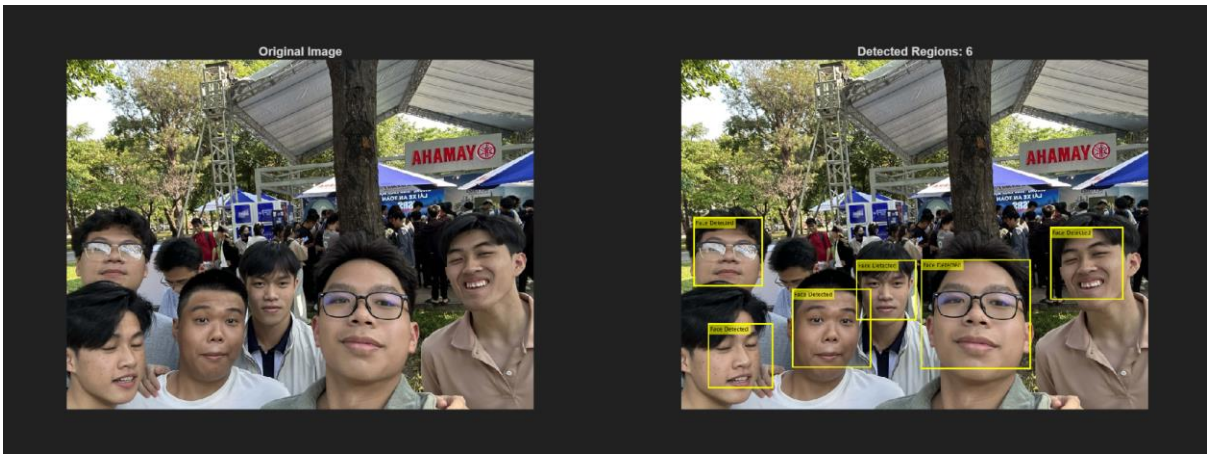
```
disp(['Number of detected regions: ' num2str(num_faces)])
```

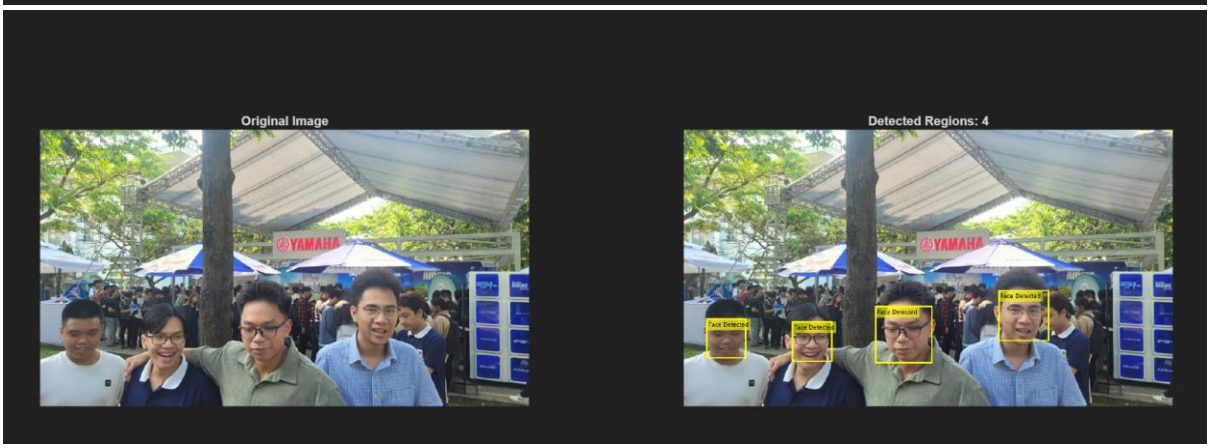
  - `disp`: Outputs text to the console
  - Useful for verifying results in text form.

2.4 Results

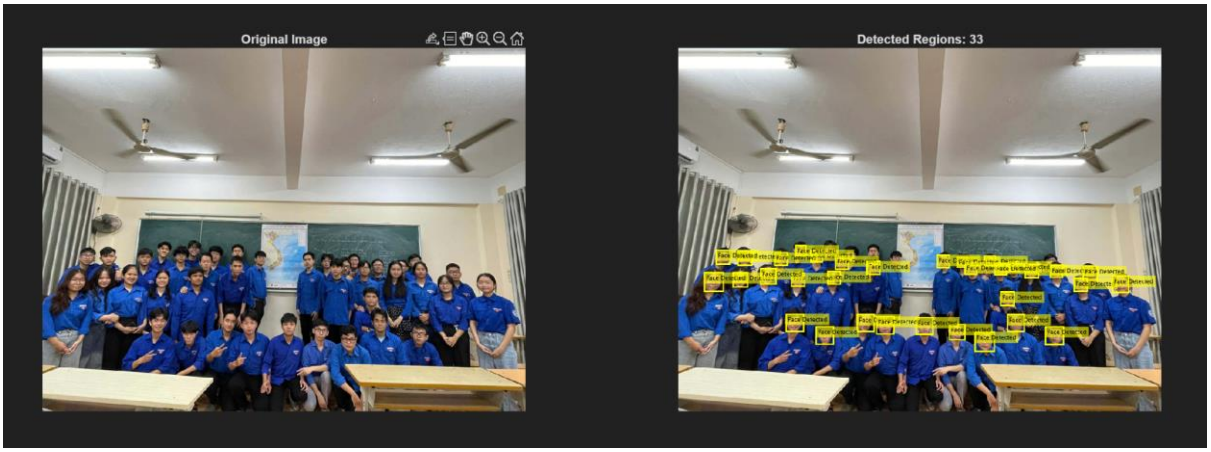
- The program was tested using several images under different conditions, including variations in lighting, face direction and image resolution.

Images:


Original Image
Detected Regions: 6
AHAMAY

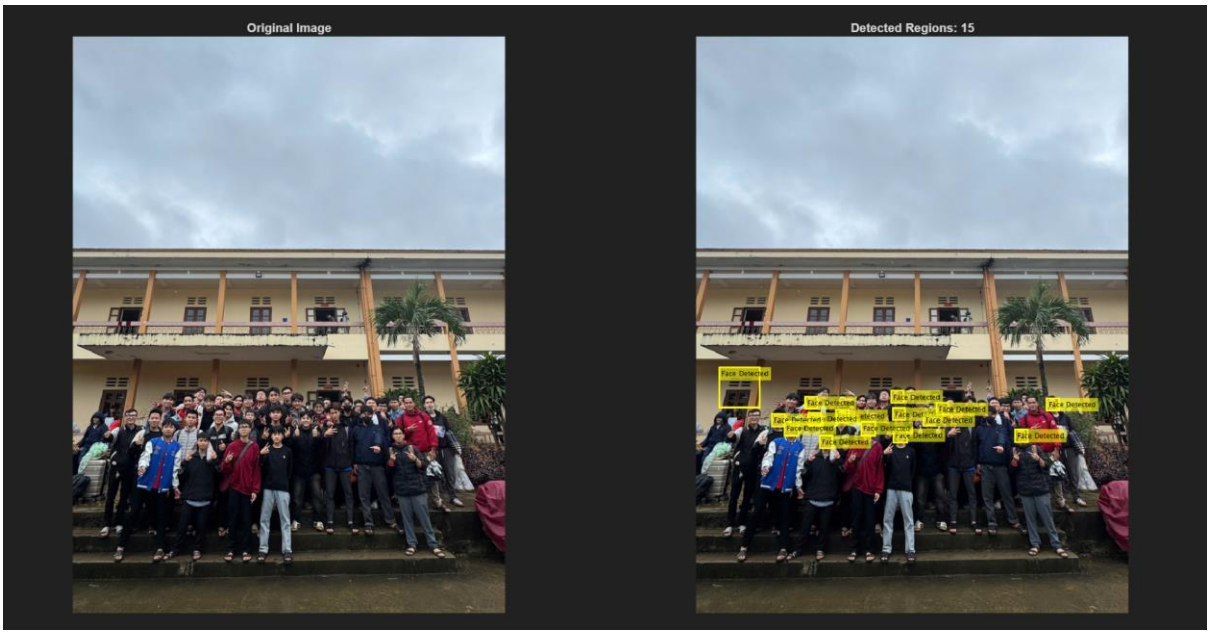

Original Image
Detected Regions: 4
YAMAHA

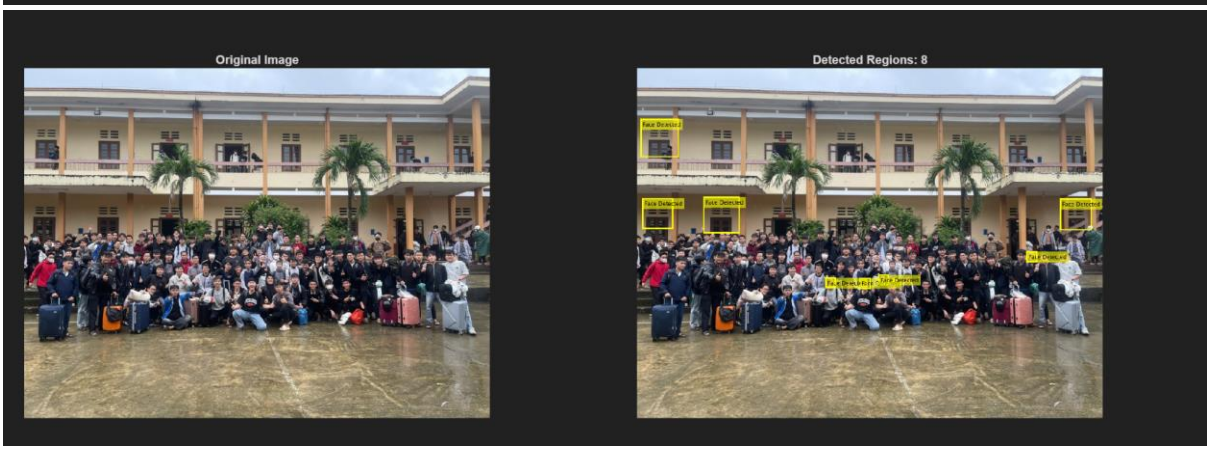
Original Image
Detected Regions: 4
FAST
FACULTY OF ADVANCED SCIENCE AND TECHNOLOGY
Innovation for Advanced Life


Original Image
Detected Regions: 33

The results demonstrate that when persons are clearly visible and facing the camera directly, the system can detect faces successfully. The detected faces are identified with rectangular boundary boxes.

However, the system still gets incorrect detection in some complex conditions such as poor lighting, too crowded environments or the face scale is too small (exceeds the MinSize limit) to effectively detect.





2.5 Advantages-Disadvantages and Future Development

2.5.1 Advantages

- The face detection program has several advantages:

+ Fast processing speed

  Using the built-in Cascade Object Detector function in MATLAB allows the system to detect faces quickly without using complex calculations.

+ Good performance for frontal faces

  The system performs well when faces are clearly visible and directly facing the camera.

+ Clear visualization

  Detected faces are highlighted using bounding boxes and rectangle labels, making the results easy to display.

+ Program design

  Each part in the system is designed as a separate function, which improves the quality in reading and debugging.

+ Helps understand the foundation of advanced applications

  The project provides basic knowledge that can be extended to more complex systems in the future.

2.5.2 Disadvantages

- The system also has several limitations besides its advantages have listed above:

+ Limited to frontal faces

  The detector works effectively for faces facing forward but may have a high chance of failure for angled faces.

+ Sensitive to lighting conditions

  Too low or high level of lighting or strong shadows can significantly reduce the accuracy.

+ False detections

  Some non-face objects may be incorrectly detected as faces if they have similar features (dark-light shadow patterns).

+ Reliance on adjusting parameters

  The detection performance depends on parameters such as MergeThreshold, ScaleFactor or MinSize.

5. Limited performance in difficult environments

   The system cannot handle properly complex conditions such as multiple angles or crowded environments in the image.

2.5.3 Future Development

- There are several directions in which the system can be improved in the future:

+ Non-frontal faces detection

  The system can be improved to detect faces at different angles in order to enhance the flexibility.

+ Using advanced algorithms

  More advanced techniques like deep learning can help improve detection accuracy and system performance.

+ Parameter optimization

  Optimizing parameters can improve detection quality for different images.

+ Real-time face detection

  The system can be extended to perform real-time face detection through web cam instead of only on images.

2.6 Conclusion

- This project successfully developed a basic face detection system using MATLAB, in which it followed the rule of Cascade Object Detector. The main contributions of this project include:

+ Developing a MATLAB program based on fundamental programming concepts.
+ Using image processing techniques like grayscale conversion and resizing.
+ Successfully detecting human faces in digital images.

The results show that the system is capable of effectively detecting frontal faces under normal conditions. It is easier to build and achieve relatively quick performance when specific built-in functions are used.

REFERENCES

[1] P. Viola and M. Jones, "Rapid object detection using a boosted cascade of simple features," in *Proceedings of the 2001 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR)*, Kauai, HI, USA, 2001, pp. I-511-I-518.

[2] ITU-R, "Studio encoding parameters of digital television for standard 4:3 and wide-screen 16:9 aspect ratios," *Recommendation ITU-R BT.601-7*, Mar. 2011.

[3] MathWorks, "Computer Vision Toolbox Documentation," *The MathWorks, Inc.*, [Online]

[4] R. C. Gonzalez and R. E. Woods, *Digital Image Processing*, 4th ed. New York, NY, USA: Pearson, 2018.